



K.N. Toosi University of Technology

OFFLINE JAVA GUIDE • BONUS

Chapter 9

JavaFX

How to build graphical desktop applications — windows, controls, layouts, events, and styling — using Java's modern GUI toolkit. Your final-project companion.

COURSE

Advanced Programming

SEMESTER

Spring 2026

DEPARTMENT

Computer Engineering

v1.0

Table of Contents

9.1	What is JavaFX?	Stage, Scene, scene graph
9.2	The Application Lifecycle	start, launch, your first window
9.3	Controls	Buttons, fields, checkboxes...
9.4	Layout Panes	VBox, HBox, BorderPane, GridPane
9.5	Event Handling	Reacting to clicks & input
9.6	Properties & Binding	Keeping the UI in sync
9.7	Styling with CSS	The -fx- stylesheet
9.8	FXML & Scene Builder	Separating UI from logic
9.9	Putting It Together	A complete mini app
9.10	Common Mistakes & Tips	Pitfalls every beginner hits

JavaFX is Java's modern toolkit for building **graphical user interfaces** (GUIs) — desktop applications with windows, buttons, text fields, and menus. It replaced the older Swing/AWT toolkits and is what you'll use for your final project.

NOTE

Throughout this chapter, the **"Rendered UI"** panels are faithful re-creations of the JavaFX windows, drawn to build your intuition. The real API differs slightly in syntax, but the visual result and every concept map one-to-one.

The Theatre Metaphor

JavaFX borrows its vocabulary from the theatre, which makes the three core objects easy to remember:

OBJECT	THEATRE ANALOGY	IN JAVAFX
Stage	The stage itself	The top-level window (with title bar, borders)
Scene	The current scene being performed	The content shown inside the window
Node	The actors and props	Every visual element : buttons, labels, layouts

The Scene Graph

The nodes in a scene form a **tree** called the *scene graph*. There is one **root** node (usually a layout pane), and every other node is a child somewhere beneath it:

```
1 Stage (the window)
2   └─ Scene (the content)
3       └─ VBox (root layout pane)
4           ├── Label "Hello, JavaFX!"
5           ├── Button "Click me"
6           └─ HBox (a nested layout)
7               ├── Button "OK"
8               └─ Button "Cancel"
```

STRUCTURE

A **Node** is any single element. A **Parent** is a node that can contain children (all layout panes are Parents). A **Control** is an interactive node like a **Button** or **TextField**.

Setting Up JavaFX

Since Java 11, JavaFX is **not bundled with the JDK** — it's a separate library. In practice you add it as a dependency:

```
1 <!-- Maven: add the JavaFX controls dependency to pom.xml -->
2 <dependency>
3     <groupId>org.openjfx</groupId>
4     <artifactId>javafx-controls</artifactId>
5     <version>21</version>
6 </dependency>
```

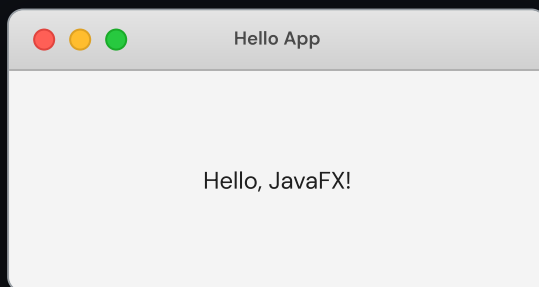
JAVA

Every JavaFX program starts by extending the `Application` class and overriding its `start()` method. This is the entry point where you build and show your first window.

Your First Window

```
1  import javafx.application.Application;
2  import javafx.scene.Scene;
3  import javafx.scene.control.Label;
4  import javafx.scene.layout.StackPane;
5  import javafx.stage.Stage;
6
7  public class HelloApp extends Application {
8
9      @Override
10     public void start(Stage stage) {
11         Label message = new Label("Hello, JavaFX!");
12
13         StackPane root = new StackPane(message);    // root layout
14         Scene scene = new Scene(root, 280, 160);    // content, 280x160 px
15
16         stage.setScene(scene);                      // put the scene in the window
17         stage.setTitle("Hello App");                // window title
18         stage.show();                               // make it visible
19     }
20
21     public static void main(String[] args) {
22         launch(args);    // starts JavaFX, which then calls start()
23     }
24 }
```

RENDERED UI



The window produced by HelloApp.

How Startup Works

1. `main()` calls `launch(args)`.
2. `launch` boots the JavaFX runtime and (optionally) calls `init()`.
3. It then calls `start(Stage)` on the JavaFX Application Thread — this is where your UI is built.
4. When the last window closes, `stop()` runs for cleanup.

METHOD	WHEN IT RUNS	USE IT FOR
<code>init()</code>	Before the UI is shown	Light setup (no UI code here)
<code>start(Stage)</code>	To build and show the window	Creating scenes, nodes, showing the stage
<code>stop()</code>	When the app is closing	Saving data, releasing resources

WARNING

All UI code must run on the **JavaFX Application Thread**. `start()` already runs there, so you're fine inside it. But if you do background work on another thread (Chapter 8!), you cannot touch the UI directly from it — you must hand it back with `Platform.runLater(...)`. More on this in section 9.10.

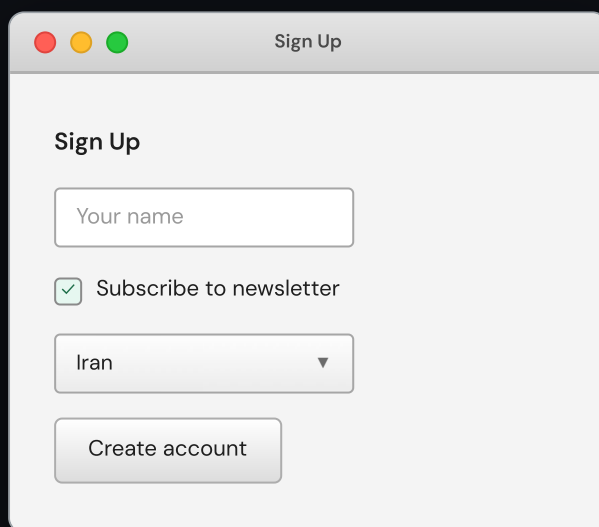
Controls are the interactive nodes users click and type into. You create one, configure it, and add it to a layout. Here are the ones you'll use most.

Building a Small Form

```
1  Label title = new Label("Sign Up");
2
3  TextField nameField = new TextField();
4  nameField.setPromptText("Your name");           // grey placeholder text
5
6  CheckBox subscribe = new CheckBox("Subscribe to newsletter");
7  subscribe.setSelected(true);
8
9  ChoiceBox<String> country = new ChoiceBox<>();
10 country.getItems().addAll("Iran", "Germany", "Japan");
11 country.setValue("Iran");                       // default selection
12
13 Button submit = new Button("Create account");
```

JAVA

RENDERED UI

A screenshot of a Java Swing window titled "Sign Up". The window has a standard macOS-style title bar with red, yellow, and green buttons. Inside the window, the title "Sign Up" is centered at the top. Below the title, there is a text input field with the placeholder text "Your name". Underneath the text field is a checked checkbox with the label "Subscribe to newsletter". Below the checkbox is a choice box (dropdown menu) with "Iran" selected and a downward arrow on the right. At the bottom of the form is a button labeled "Create account".

A form built from Label, TextField, CheckBox, ChoiceBox, and Button.

Reading & Changing a Control's Value

```
1  String typedName = nameField.getText();         // read the text field
2  boolean wantsEmail = subscribe.isSelected();    // read the checkbox
3  String chosen = country.getValue();              // read the choice box
4
5  nameField.setText("Sara");                       // set a value in code
6  submit.setDisable(true);                         // grey out the button
```

JAVA

Common Controls

CONTROL	PURPOSE	KEY METHODS
<code>Label</code>	Non-editable text	<code>setText</code> , <code>getText</code>
<code>Button</code>	Clickable action	<code>setOnAction</code> , <code>setDisable</code>
<code>TextField</code>	Single-line input	<code>getText</code> , <code>setPromptText</code>
<code>TextArea</code>	Multi-line input	<code>getText</code> , <code>setWrapText</code>
<code>CheckBox</code>	On/off toggle	<code>isSelected</code> , <code>setSelected</code>
<code>RadioButton</code>	One choice from a group	<code>setToggleGroup</code> , <code>isSelected</code>
<code>ChoiceBox</code> / <code>ComboBox</code>	Drop-down selection	<code>getItems</code> , <code>getValue</code>
<code>Slider</code>	Pick a number in a range	<code>getValue</code> , <code>valueProperty</code>

NOTE

Radio buttons only become mutually exclusive when you add them to a shared `ToggleGroup` . Without it, the user could select all of them at once. `ChoiceBox` is best for a few fixed options; `ComboBox` adds features like editable text and larger lists.

You never position controls by pixel coordinates. Instead you put them inside a **layout pane**, and the pane arranges its children for you. Choosing the right pane is the key skill in GUI design.

VBox — a vertical stack

```
1 VBox root = new VBox(10);           // 10 px spacing between children
2 root.setPadding(new Insets(15));     // 15 px padding around the edges
3 root.setAlignment(Pos.CENTER);
4 root.getChildren().addAll(
5     new Label("Name"), new TextField(), new Button("Save")
6 );
```

JAVA

RENDERED UI

VBOX — STACKS CHILDREN TOP TO BOTTOM



HBox — a horizontal row

```
1 HBox buttons = new HBox(10);
2 buttons.setAlignment(Pos.CENTER_RIGHT);
3 buttons.getChildren().addAll(new Button("Cancel"), new Button("OK"));
```

JAVA

RENDERED UI

HBOX — ARRANGES CHILDREN LEFT TO RIGHT



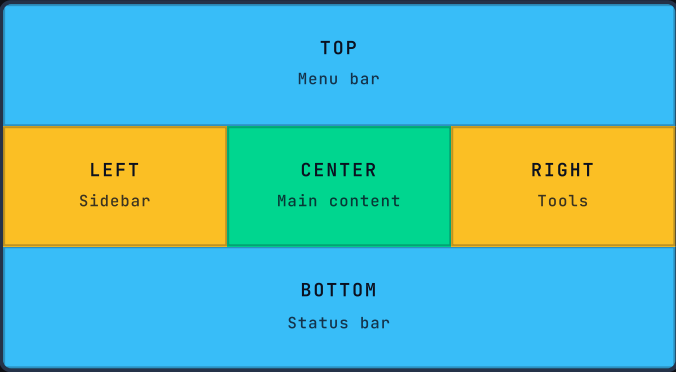
BorderPane — five regions

Perfect for a classic app shell: a toolbar on top, a sidebar on the left, content in the middle, a status bar at the bottom. The center expands to fill remaining space.

```

1  BorderPane root = new BorderPane();
2  root.setTop(new Label("Menu bar"));
3  root.setLeft(new Label("Sidebar"));
4  root.setCenter(new Label("Main content"));
5  root.setRight(new Label("Tools"));
6  root.setBottom(new Label("Status bar"));

```



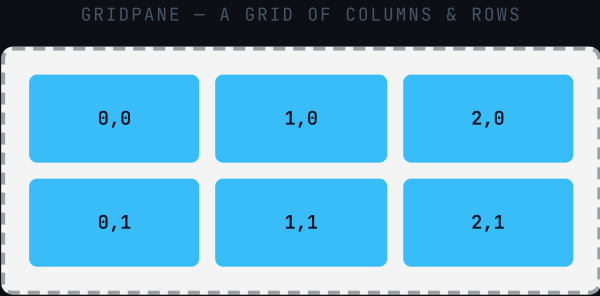
GridPane — rows and columns

Ideal for forms. You add each node at a `(column, row)` coordinate:

```

1  GridPane grid = new GridPane();
2  grid.setHgap(8);
3  grid.setVgap(8);
4  grid.add(new Label("Name:"), 0, 0); // column 0, row 0
5  grid.add(new TextField(), 1, 0); // column 1, row 0
6  grid.add(new Label("Email:"), 0, 1); // column 0, row 1
7  grid.add(new TextField(), 1, 1); // column 1, row 1

```



PANE	ARRANGES CHILDREN...
<code>VBox</code>	In a single vertical column
<code>HBox</code>	In a single horizontal row
<code>BorderPane</code>	Into top / left / center / right / bottom
<code>GridPane</code>	In a flexible grid of rows and columns
<code>StackPane</code>	Stacked on top of each other (layered/centered)
<code>FlowPane</code>	Left to right, wrapping to the next line as needed

TIP

Panes **nest**. A real window is usually a `BorderPane` whose center is a `VBox` that contains an `HBox` of buttons, and so on. Build complex layouts by composing simple panes rather than reaching for absolute pixel positions.

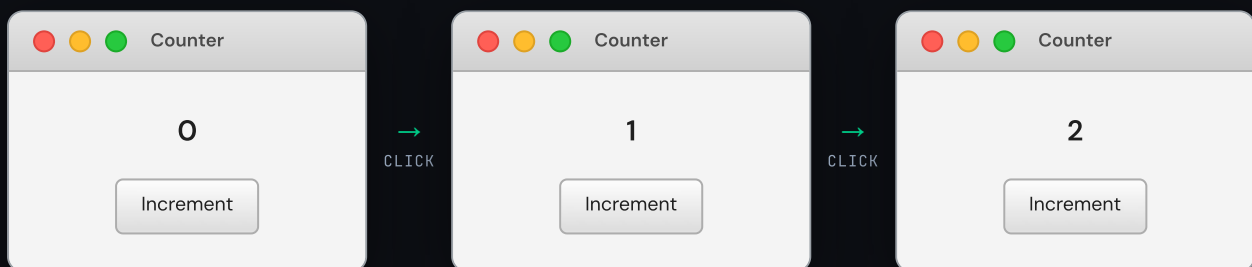
A GUI is **event-driven**: your code sits idle until the user does something — a click, a keypress — which fires an **event**. You attach a **handler** (a lambda from Chapter 7!) that runs in response.

Handling a Button Click

```
1  Label count = new Label("0");
2  Button increment = new Button("Increment");
3
4  // setOnAction takes an EventHandler — a functional interface, so use a lambda
5  increment.setOnAction(e → {
6      int value = Integer.parseInt(count.getText());
7      count.setText(String.valueOf(value + 1));
8  });
```

JAVA

Because a PDF can't be clicked, here's the behavior shown as successive frames — each click runs the handler and updates the label:



RENDERED UI

The label after zero, one, and two clicks.

The Event Object

The handler receives an event object carrying details — which node fired it, mouse coordinates, which key was pressed:

```
1  button.setOnAction(e → {
2      System.out.println("Source: " + e.getSource());    // the Button
3  });
4
5  // Mouse and keyboard events too
6  node.setMouseClicked(e → System.out.println(e.getX() + ", " + e.getY()));
7  field.setOnKeyPressed(e → System.out.println("Key: " + e.getCode()));
```

JAVA

METHOD	FIRES WHEN...
<code>setOnAction</code>	A button is clicked / an action is triggered
<code>setMouseClicked</code>	The node is clicked with the mouse
<code>setOnMouseEntered</code> / <code>Exited</code>	The pointer hovers over / leaves the node
<code>setOnKeyPressed</code>	A key is pressed while the node has focus

Keep event handlers **short**. A handler runs on the JavaFX Application Thread, so any slow work inside it (reading a big file, a network call) *freezes the entire UI* until it finishes. Move heavy work to a background thread and update the UI when done — see 9.10.

Most JavaFX values are **observable properties** — objects that notify listeners whenever they change. This lets you keep parts of the UI **automatically in sync** without writing update code by hand.

Listening to a Property

```
1 Slider slider = new Slider(0, 100, 0); // min, max, start
2 Label readout = new Label("0");
3
4 // React whenever the slider's value changes
5 slider.valueProperty().addListener((obs, oldVal, newVal) → {
6     readout.setText("Value: " + newVal.intValue());
7 });
```

JAVA

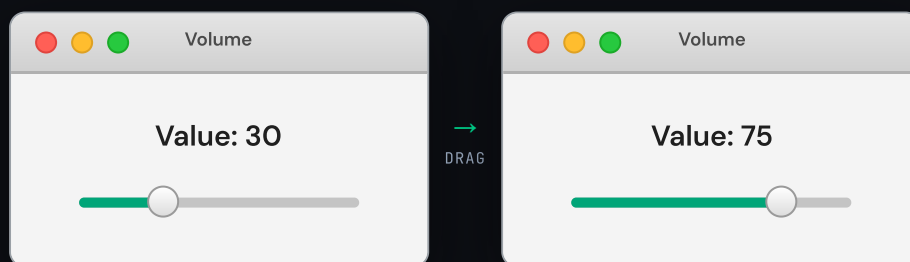
Binding — even simpler

A **binding** ties one property directly to another, so it updates itself. No listener, no handler — just declare the relationship once:

```
1 Label readout = new Label();
2
3 // readout's text is now permanently tied to the slider's value
4 readout.textProperty().bind(slider.valueProperty().asString("Value: %.0f"));
```

JAVA

Dragging the slider updates the label instantly. Shown as two states:



RENDERED UI

The bound label tracks the slider automatically.

APPROACH

USE WHEN...

```
property.addListener(...)
```

You need to *run logic* on each change

```
a.bind(b)
```

You want **a** to always *equal* some function of **b**

```
a.bindBidirectional(b)
```

Two editable fields should mirror each other

MISTAKE

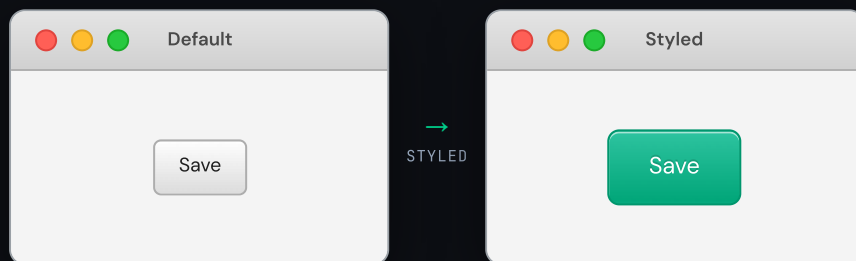
Once a property is **bound**, you can no longer set it directly — `readout.setText("x")` throws `RuntimeException: A bound value cannot be set.` Either let the binding drive it, or call `unbind()` first if you need manual control again.

JavaFX is styled with **CSS** — the same idea as web CSS, but every property is prefixed with `-fx-`. You can style inline, or (better) in a separate stylesheet.

Inline Styling

```
1 Button button = new Button("Save");
2 button.setStyle(
3     "-fx-background-color: #00a578;" +
4     "-fx-text-fill: white;" +
5     "-fx-font-size: 14px;" +
6     "-fx-padding: 8 20;" +
7     "-fx-background-radius: 6;"
8 );
```

JAVA



RENDERED UI

The same Button before and after applying an `-fx-` style.

External Stylesheet (recommended)

Keep styling out of your Java code. Put rules in a `.css` file and attach it to the scene:

```
1  /* file: styles.css */
2  .button {
3      -fx-background-color: #00a578;
4      -fx-text-fill: white;
5      -fx-background-radius: 6;
6  }
7  .button:hover {
8      -fx-background-color: #00c98f;  /* changes on mouse-over */
9  }
10 #title-label {
11     -fx-font-size: 22px;
12     -fx-font-weight: bold;
13 }
```

CSS

```
1 // Attach the stylesheet to the scene
2 scene.getStylesheets().add("styles.css");
3
4 // Target a rule by style class or id
5 button.getStyleClass().add("button"); // matches .button
6 title.setId("title-label"); // matches #title-label
```

PROPERTY	EFFECT
<code>-fx-background-color</code>	Fill color of the node
<code>-fx-text-fill</code>	Text color
<code>-fx-font-size</code> / <code>-fx-font-weight</code>	Text size / boldness
<code>-fx-padding</code>	Inner spacing
<code>-fx-background-radius</code>	Rounded corners
<code>-fx-border-color</code> / <code>-fx-border-width</code>	Border styling

TIP

Prefer an external stylesheet with **style classes** over inline `setStyle`. It keeps your Java focused on logic, lets you restyle the whole app in one place, and supports states like `:hover` that inline styles can't express.

Building a big UI entirely in Java gets messy. **FXML** is an XML format that describes the layout *declaratively*, separating the **look** (FXML) from the **logic** (a Java controller). This is the recommended structure for a real project.

The Layout — a .fxml file

```

1  <?import javafx.scene.control.*?>
2  <?import javafx.scene.layout.*?>
3
4  <VBox spacing="10" alignment="CENTER"
5      xmlns:fx="http://javafx.com/fxml"
6      fx:controller="com.example.GreetController">
7
8      <TextField fx:id="nameField" promptText="Your name"/>
9      <Button text="Greet" onAction="#handleGreet"/>
10     <Label fx:id="output"/>
11 </VBox>

```

FXML

The Logic — a controller class

Fields marked `@FXML` are wired to nodes with a matching `fx:id`; methods named in `onAction` become event handlers:

```

1  public class GreetController {
2
3      @FXML private TextField nameField;    // linked to fx:id="nameField"
4      @FXML private Label output;          // linked to fx:id="output"
5
6      @FXML
7      private void handleGreet() {         // linked to onAction="#handleGreet"
8          String name = nameField.getText();
9          output.setText("Hello, " + name + "!");
10     }
11 }

```

JAVA

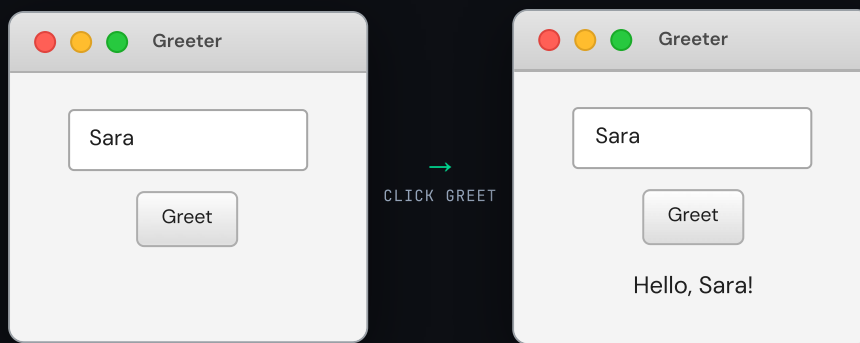
Loading the FXML

```

1  @Override
2  public void start(Stage stage) throws Exception {
3      Parent root = FXMLLoader.load(getClass().getResource("greet.fxml"));
4      stage.setScene(new Scene(root, 280, 180));
5      stage.setTitle("Greeter");
6      stage.show();
7  }

```

JAVA



The FXML view driven by its controller.

NOTE

Scene Builder is a free drag-and-drop tool that generates FXML visually — you design the window by dragging controls onto a canvas, and it writes the `.fxml` for you. You still write the controller by hand. It pairs perfectly with FXML for larger projects.

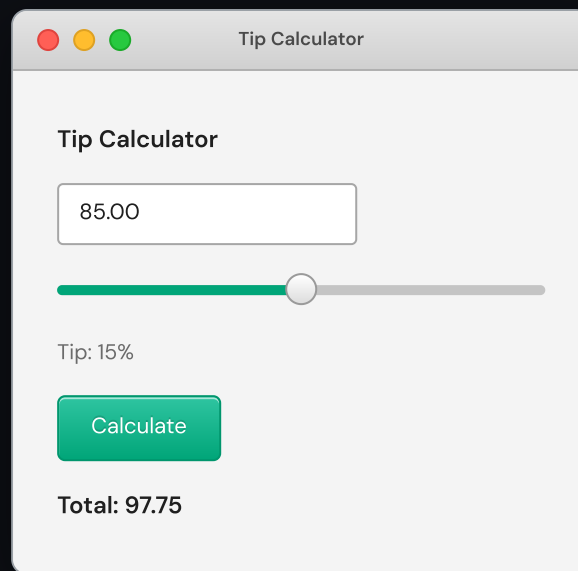
MISTAKE

The most common FXML bug: an `@FXML` field is `null` at runtime. Causes are almost always a mismatch — the `fx:id` in the FXML doesn't exactly match the field name, the `fx:controller` points to the wrong class, or you tried to use a node *before* the FXML finished loading. Check that names line up character-for-character.

Let's combine everything — controls, a layout, an event handler, and a bit of logic — into one complete, runnable app: a simple **tip calculator**.

```
1  public class TipApp extends Application {
2
3      @Override
4      public void start(Stage stage) {
5          Label title    = new Label("Tip Calculator");
6          TextField bill = new TextField();
7          bill.setPromptText("Bill amount");
8
9          Slider tip = new Slider(0, 30, 15);    // 0-30%, start at 15
10         tip.setShowTickLabels(true);
11         Label tipLabel = new Label("Tip: 15%");
12         tip.valueProperty().addListener((o, ov, nv) →
13             tipLabel.setText("Tip: " + nv.intValue() + "%"));
14
15         Button calc  = new Button("Calculate");
16         Label result = new Label();
17
18         calc.setOnAction(e → {
19             double amount = Double.parseDouble(bill.getText());
20             double total  = amount * (1 + tip.getValue() / 100);
21             result.setText(String.format("Total: %.2f", total));
22         });
23
24         VBox root = new VBox(12, title, bill, tip, tipLabel, calc, result);
25         root.setPadding(new Insets(20));
26         root.setAlignment(Pos.CENTER_LEFT);
27
28         stage.setScene(new Scene(root, 300, 260));
29         stage.setTitle("Tip Calculator");
30         stage.show();
31     }
32
33     public static void main(String[] args) {
34         launch(args);
35     }
36 }
```

JAVA



The finished tip calculator — controls, layout, binding, and an event handler working together.

TIP

This is the shape of most JavaFX apps: create controls, wire their events to logic, arrange them in a pane, drop the pane in a scene, and show the stage. For your final project, start this small and grow it — add validation, split the UI into FXML, and move styling into a stylesheet.

1. Forgetting to Show the Stage

```

1  // BUG – the window is built but never appears
2  stage.setScene(scene);
3  // ... nothing happens, no window
4
5  // FIX – you must call show()
6  stage.setScene(scene);
7  stage.show();

```

JAVA

2. Updating the UI From a Background Thread

```

1  // BUG – touching a control off the JavaFX thread throws IllegalStateException
2  new Thread(() → {
3      String data = downloadData();      // slow work – OK on a background thread
4      label.setText(data);                // WRONG – UI update off the FX thread!
5  }).start();
6
7  // FIX – hand the UI update back to the JavaFX Application Thread
8  new Thread(() → {
9      String data = downloadData();
10     Platform.runLater(() → label.setText(data)); // correct
11 }).start();

```

JAVA

WARNING

This is the #1 crash when JavaFX meets Chapter 8's threads. **Rule:** do slow work on a background thread, but wrap every UI change in `Platform.runLater(...)`. For structured background tasks, JavaFX also provides the `Task` class, which reports progress and results safely.

3. Blocking the UI Thread

```

1  // BUG – Thread.sleep on the FX thread FREEZES the whole window
2  button.setOnAction(e → {
3      Thread.sleep(5000);      // UI is frozen for 5 seconds!
4      label.setText("Done");
5  });
6  // FIX – never do long/blocking work inside an event handler; use a thread

```

JAVA

4. Adding One Node to Two Parents

```

1  Button b = new Button("OK");
2  vbox.getChildren().add(b);
3  hbox.getChildren().add(b);    // BUG – a node can have only ONE parent
4  // The button silently moves to hbox and vanishes from vbox
5  // FIX – create a separate node for each place it appears

```

JAVA

5. Parsing Text Without Handling Bad Input

JAVA

```
1 // BUG - if the field is empty or not a number, this throws
2 double amount = Double.parseDouble(bill.getText());
3
4 // FIX - validate and handle the error (Chapter 6!)
5 try {
6     double amount = Double.parseDouble(bill.getText());
7     // ... use amount
8 } catch (NumberFormatException ex) {
9     result.setText("Please enter a valid number");
10 }
```

6. Setting a Bound Property by Hand

JAVA

```
1 label.textProperty().bind(slider.valueProperty().asString());
2 label.setText("hello"); // BUG - RuntimeException: bound value cannot be set
3 // FIX - call label.textProperty().unbind() first, or let the binding drive it
```

QUICK REFERENCE - JAVAFX ESSENTIALS

- › Extend `Application`, override `start(Stage)`, call `launch(args)` in `main`
- › The chain is always **Stage** → **Scene** → **root pane** → **nodes**; finish with `stage.show()`
- › Arrange nodes with layout panes (VBox / HBox / BorderPane / GridPane), never pixel coordinates
- › Wire behavior with `setOnAction(e → ...)` and keep handlers short
- › Use properties & `bind()` to keep the UI in sync automatically
- › Style with an external `-fx-` stylesheet and style classes
- › For bigger UIs, split into **FXML + a controller** (Scene Builder helps)

QUICK REFERENCE - THE THREADING RULE

- › All UI code runs on the **JavaFX Application Thread**
- › Never do slow work (files, network, sleep) in an event handler — it freezes the window
- › Run slow work on a background thread, then update the UI via `Platform.runLater(...)`
- › Consider the JavaFX `Task` class for background jobs that report progress or a result

This document was compiled from the following resources. All content was adapted and restructured for educational use. The "Rendered UI" panels are HTML re-creations for illustration.

[1] **OpenJFX — Official JavaFX Documentation**

Getting started, Application lifecycle, controls, layouts, FXML, CSS
<https://openjfx.io/openjfx-docs/>

[2] **Oracle — JavaFX API Documentation**

Stage, Scene, Node, controls, layout panes, properties & bindings
<https://openjfx.io/javadoc/21/>

[3] **Jenkov — JavaFX Tutorial**

Controls, layouts, event handling, CSS styling, FXML, Scene Builder
<https://jenkov.com/tutorials/javafx/index.html>

[4] **GeeksforGeeks — JavaFX**

Application structure, common controls, layout panes, examples
<https://www.geeksforgeeks.org/javafx-tutorial/>

[5] **Oracle — JavaFX: Working with the JavaFX Scene Graph & Concurrency**

Scene graph, the Application Thread, Platform.runLater, the Task class
<https://docs.oracle.com/javase/8/javafx/interoperability-tutorial/concurrency.htm>

CREATED BY

Sepehr Ghardashi

sepehr.ghardashi@gmail.com